

Mycellius — Séance 5 (4h) :

Persistence DB + migrations

*Modèle DB (MCD/MLD) + MySQL + JPA + hibernate + migrations (Flyway/Liquibase)
+ branchement endpoints C1/C2 sur DB + sauvegarde/restoration*

Objectif de la séance

À la fin de cette séance, tu dois avoir :

- un modèle de données simple pour les pages du wiki (MCD/MLD)
- une base MySQL qui tourne dans un conteneur Docker pour l'environnement de développement
- des migrations Flyway versionnées qui créent le schéma de base (table des pages, éventuellement une deuxième migration d'exemple)
- Spring Boot configuré pour parler à MySQL avec JPA/Hibernate
- les endpoints C1/C2 de séance 4 (/api/v1/pages ...) branchés sur la DB (création/lecture persistantes)
- une mini procédure de sauvegarde/restoration (mysqldump + restore) testée sur ta base de dev
- la CI GitHub toujours verte (mvn -f api/pom.xml test), sans dépendre d'une DB dans GitHub Actions

Découpage indicatif de la séance

1. Rappel séance 4 et objectif séance 5 (10–15 min)
2. Modèle de données : MCD / MLD pour les pages (30–40 min)
3. Mettre MySQL dans Docker (dev) (40–50 min)
4. Configurer Spring Boot avec JPA + Flyway (40–50 min)
5. Créer les migrations versionnées (Flyway) (30–40 min)
6. Brancher WikiService et les endpoints C1/C2 sur la DB (40–50 min)
7. Sauvegarde / restauration de la DB de dev (20–30 min)
8. CI + workflow Git dev → test (20–30 min)

1. Rappel séance 4 et positionnement de la séance 5

1.1 Ce que tu as à la fin de la séance 4

Dans le module api :

- MycelliusApiApplication qui démarre Spring Boot sur <http://localhost:8080>
- un endpoint technique /api/health pour vérifier que l'API répond
- un contrôleur REST (WikiPageController) avec au moins :
 - POST /api/v1/pages pour créer une page en mémoire
 - GET /api/v1/pages/{id} pour lire une page par id
 - éventuellement GET /api/v1/pages/search?title=fragment pour rechercher par titre
- un noyau métier déjà existant :
 - Tag, WikiPage
 - InMemoryWiki (Map<String, WikiPage>)
 - WikiService (createPage, getPageById, searchByTitle...)
- des exceptions métier : IllegalArgumentException, PageNotFoundException
- des tests JUnit sur le modèle et sur WikiService
- une CI GitHub qui lance mvn -f api/pom.xml test sur les branches dev / test / prod

Idée importante :

Jusqu'ici, tout se passe "en mémoire". Quand tu redémarres l'appli, les pages disparaissent. La séance 5 consiste à brancher ce que tu as déjà sur une vraie base MySQL, sans jeter le travail précédent.

1.2 Ce qu'on vise dans cette séance

- concevoir un schéma relationnel minimal aligné avec WikiPage
- déployer une DB MySQL dans Docker (environnement de développement)
- laisser Flyway créer/modifier ce schéma via des scripts versionnés
- utiliser JPA / Spring Data pour accéder aux données depuis le code Java
- faire en sorte que POST / GET sur les pages manipulent vraiment la DB
- préparer le terrain pour la séance 6 (DTO, mappers, pagination, tags)

2. Modèle de données : MCD / MLD pour les pages

2.1 Rappel du modèle objet actuel

Aujourd'hui, côté Java, tu as au minimum :

- WikiPage avec des champs du type id (String), title (String), content (String ou texte long), et éventuellement une liste de Tag
- Tag avec au moins une valeur normalisée (String value)

Pour la persistance, on va commencer par quelque chose de simple :

- stocker les pages dans une table WIKI_PAGE
- ignorer pour l'instant la relation "Tag ↔ WikiPage" ou la représenter de façon simplifiée (les tags seront traités proprement en séance 6).

2.2 MCD minimal

Sur un schéma conceptuel (MCD), tu peux représenter :

- Entité PAGE
 - id_page (identifiant métier, par exemple "PAGE-001")
 - title
 - content
 - created_at (date/heure de création)

Plus tard, tu ajouteras une entité TAG et une association PAGE_TAG, mais ce n'est pas indispensable pour commencer la persistance.

2.3 MLD minimal

Traduction en modèle logique de données (MLD) pour MySQL :

- Table WIKI_PAGE
 - id VARCHAR(64) PRIMARY KEY
 - title VARCHAR(255) NOT NULL
 - content TEXT NOT NULL
 - created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP

Le MCD (Modèle Conceptuel de Données) et le MLD (Modèle Logique de Données) décrivent les mêmes informations, mais à deux niveaux différents d'abstraction.

Le **MCD correspond à la vision "métier"**, indépendante de toute technologie. Il sert à décrire ce dont l'entreprise a besoin de mémoriser : les entités (ex. PAGE), leurs attributs (title, content, created_at) et les associations entre entités (ex. plus tard PAGE-TAG). À ce stade, on ne parle pas de tables, ni de types SQL, ni de clés techniques propres à un SGBD ; l'objectif est de valider le sens, le périmètre fonctionnel, et les règles de gestion.

Le **MLD est la traduction du MCD vers un modèle relationnel exploitable par une base de données**. On passe à une vision "implémentation logique" : tables, colonnes, clés primaires et étrangères, contraintes (NOT NULL, UNIQUE), et préparation des relations (1-N, N-N via table d'association). On reste encore assez "portable", mais on commence à tenir compte des choix du monde relationnel (par exemple, une association N-N devient une table intermédiaire).

En résumé : MCD = "quoi et pourquoi" (métier, indépendant du SGBD) ; MLD = "comment le stocker en relationnel" (tables et contraintes, prêt à être décliné ensuite en SQL).

Exemple direct sur notre cas : en MCD tu as l'entité PAGE avec id_page, title, content, created_at. En MLD, tu la matérialises en table WIKI_PAGE avec une clé primaire (id), des types SQL, et des contraintes. Et quand tu ajouteras TAG : en MCD tu auras une association PAGE-TAG (souvent N,N), et en MLD cela deviendra une table d'association (PAGE_TAG) avec deux clés étrangères (id_page, id_tag), souvent clé primaire composite.

En résumé : MCD = "quoi" (conceptuel/métier, indépendant technique), MLD = "comment" (structure relationnelle prête à implémenter dans MySQL).

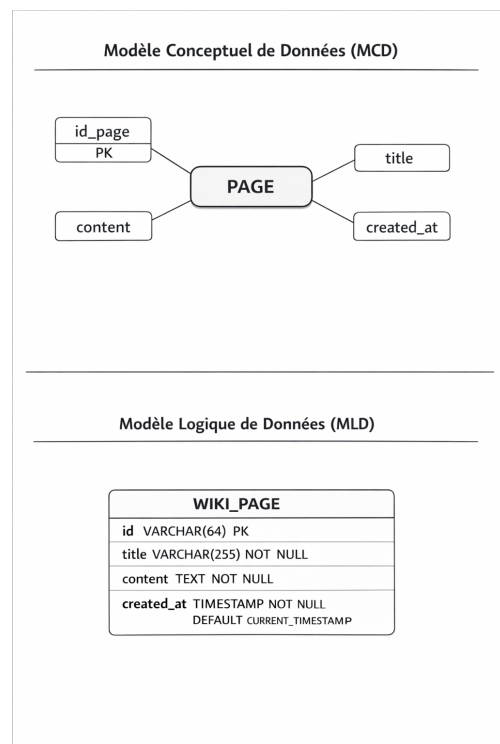
Optionnel :

- contrainte d'unicité sur title si tu veux interdire deux titres identiques :

- UNIQUE(title)

Idée :

- On commence par une seule table, très lisible.
- On garde les tags en mémoire ou sous une forme texte simple si besoin.
- L'important est de bien comprendre le chemin complet :



requête HTTP → contrôleur Spring → service → repository → DB MySQL.

3. Mettre MySQL dans Docker (environnement de développement)

Objectif

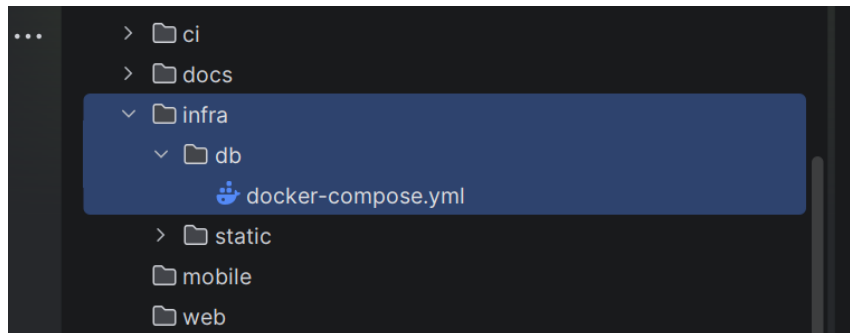
Avoir une DB MySQL qui tourne sur ta machine (ou ta VM dev) sans l'installer "à la main", uniquement via Docker.

3.1 Créer un docker-compose pour MySQL

Dans ton dépôt mycellius/, crée un sous-dossier dédié, par exemple infra/db.

Attention, le dossier infra a déjà été créé... si, si, regarde bien :). donc, crée un sous-dossier db puis dedans crée un fichier docker-compose comme indiqué ci-dessous.

Fichier : infra/db/docker-compose.yml



Contenu proposé (à adapter si tu as déjà quelque chose) :

```
version: "3.9"
services:
  mysql:
    image: mysql:8.0
    container_name: mycellius-mysql
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: rootpassword
      MYSQL_DATABASE: mycellius
      MYSQL_USER: mycellius
      MYSQL_PASSWORD: mycellius
    ports:
      - "3306:3306"
    volumes:
      - ./data:/var/lib/mysql
```

Commentaires :

- **image mysql:8.0** : version moderne de MySQL.
- **port 3306:3306** : MySQL sera accessible sur localhost:3306.
- **MYSQL_DATABASE / USER / PASSWORD** : DB et utilisateur applicatif.
- **volume ./data** : les données persistent même si tu redémarres le conteneur.

3.2 Démarrer MySQL avec Docker

Avant de "up" ton docker, tu dois d'abord savoir si tu as installé Docker en local sur ton poste... je te laisse chercher un peu... et après, continue avec ce qui suit.

Depuis la racine du dépôt (C:\Users\...\mycellius) :

```
cd infra/db
docker compose up -d
```

```
PS C:\Users\clakg\mycellius\infra\db> docker compose up -d
time="2026-01-13T00:08:38+01:00" level=warning msg="C:\\Users\\clakg\\mycellius\\infra\\db\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 16/16
✓ Image mysql:8.0 Pulled 41.0s
✓ Network db_default Created 0.1s
✓ Container mycellius-mysql Created 0.6s
```

Vérifier que le conteneur est bien démarré :

```
docker ps
```

```
PS C:\Users\clakg\mycellius\infra\db> docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
7c6d7d589ccf   mysql:8.0   "docker-entrypoint.s..." About a minute ago Up About a minute   0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp   mycellius-mysql
```

=> Tu dois voir un conteneur mycellius-mysql avec le port 3306 publié.

3.3 Premier test de connexion à MySQL

Toujours dans un terminal :

```
docker exec -it mycellius-mysql mysql -u mycellius -pmycellius
```

Une fois dans le client mysql :

```
SHOW DATABASES;
USE mycellius;
SHOW TABLES;
```

=> Pour l'instant, **SHOW TABLES** doit être vide : Flyway créera la table plus tard (schéma ci-dessous).

```
PS C:\Users\clakg\mycellius\infra\db> docker exec -it mycellius-mysql mysql -u mycellius -p mycellius
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 8.0.44 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| mycellius |
| performance_schema |
+-----+
3 rows in set (0.00 sec)

mysql> USE mycellius;
Database changed
mysql> SHOW TABLES;
Empty set (0.00 sec)
```

4. Configurer Spring Boot avec JPA + Flyway + MySQL

Objectif

Brancher le module api sur la DB MySQL via JPA/Hibernate et préparer Flyway pour les migrations.

4.1 Ajouter les dépendances dans api/pom.xml

Dans le pom.xml du module api, dans la section <dependencies>, ajoute :

Dépendance JPA / Spring Data :

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

Driver MySQL :

```
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <scope>runtime</scope>
</dependency>
```

Flyway (migrations DB) :

```
<dependency>
  <groupId>org.flywaydb</groupId>
  <artifactId>flyway-core</artifactId>
</dependency>
```

Rappel des rôles :

- spring-boot-starter-data-jpa apporte Hibernate et Spring Data JPA (repos, mapping objet-relationnel).
- mysql-connector-j est le driver JDBC MySQL.
- flyway-core permet de lancer automatiquement des scripts SQL versionnés au démarrage.

=> Tu auras surement des warnings dans ton pom.xml, je te laisse débbugger. N'hésite pas à me prévenir quand tu seras rendu là ou si tu bloques

4.2 Configurer application.properties pour la connexion MySQL

On va créer le fichier de config Spring Boot et crée le fichier suivant.

Spring Boot sera content : il saura quel driver utiliser et à quelle base se connecter.

Dans api/src/main/resources, crée (ou complète) application.properties :

```
spring.datasource.url=jdbc:mysql://localhost:3306/mycellius?  
useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC  
spring.datasource.username=mycellius  
spring.datasource.password=mycellius  
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

```
spring.jpa.hibernate.ddl-auto=none  
spring.jpa.show-sql=true
```

```
spring.flyway.enabled=true  
spring.flyway.locations=classpath:db/migration
```

Explications essentielles :

- **spring.datasource.*** : paramètres pour se connecter à MySQL dans Docker.
- **ddl-auto=none** : on ne laisse pas Hibernate créer ou modifier le schéma ; ce sera le travail de Flyway.
- **show-sql=true** : pratique pour voir les requêtes SQL générées dans la console.
- **flyway.enabled=true** + locations : Spring Boot va chercher des scripts SQL dans src/main/resources/db/migration au démarrage.

5. Créer les migrations Flyway (schéma versionné)

Objectif

Créer la table WIKI_PAGE via un script de migration V1, puis montrer qu'on peut faire évoluer le schéma avec une V2.

5.1 Arborescence Flyway

Dans api/src/main/resources, crée les dossiers :

> db/migration

Tu obtiens : api/src/main/resources/db/migration

5.2 Migration V1 : création du schéma minimal

Fichier : api/src/main/resources/db/migration/V1__create_wiki_page_table.sql

Contenu proposé :

```
CREATE TABLE IF NOT EXISTS wiki_page (  
    id VARCHAR(64) PRIMARY KEY,  
    title VARCHAR(255) NOT NULL,  
    content TEXT NOT NULL,  
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

Commentaires :

- V1__... : Flyway lit le préfixe V1 comme "version 1".
- IF NOT EXISTS évite une erreur si la table existe déjà.

5.3 Migration V2 (optionnelle) : ajouter une colonne ou une contrainte

Pour illustrer l'évolution versionnée, tu peux ajouter une deuxième migration.

Exemple : ajouter un index sur le titre pour accélérer la recherche.

Fichier : V2__add_title_index.sql

```
CREATE INDEX idx_wiki_page_title ON wiki_page(title);
```

Idée :

- Tu montres que le schéma évolue par petites versions, comme le code.
- Si tu déploies sur une autre machine, il suffira de lancer l'appli pour que Flyway rejoue l'historique de migrations.

5.4 Tester les migrations

Assure-toi que le conteneur MySQL tourne (docker ps).

Puis, depuis la racine du dépôt :

```
mvn -f api/pom.xml spring-boot:run
```

Au démarrage, tu dois voir dans les logs :

- Flyway V1 ... successful
- Flyway V2 ... successful (si tu as créé V2)

Une fois l'appli démarrée, vérifie dans MySQL :

```
docker exec -it mycellius-mysql mysql -u mycellius -p mycellius
```

```
USE mycellius;
SHOW TABLES;
```

```
mysql> USE mycellius;
Database changed
mysql> SHOW TABLES;
+-----+
| Tables_in_mycellius |
+-----+
| flyway_schema_history |
| wiki_page            |
+-----+
2 rows in set (0.01 sec)

mysql>
```

=> Tu dois voir wiki_page et les tables internes de Flyway (flyway_schema_history).

6. Brancher WikiService et les endpoints C1/C2 sur la DB

Objectif

Remplacer le stockage en mémoire par un stockage en DB pour l'API, tout en conservant InMemoryWiki pour les tests unitaires.

6.1 Introduire une interface WikiRepository

Dans Spring Boot, une interface sert surtout à :

- Définir un contrat : quelles méthodes existent, sans dire comment elles sont faites.
- Réduire le couplage : le code dépend de l'interface, pas d'une classe précise (plus facile à faire évoluer).
- Faciliter les tests : on peut remplacer l'implémentation par un mock.
- Permettre plusieurs implémentations : ex. paiement Stripe ou PayPal.
- Spring peut générer l'implémentation : notamment pour les Repository JPA.

En gros:

Une interface, c'est un contrat.

Elle définit ce qu'une classe doit savoir faire, mais pas comment elle le fait.

Elle contient des signatures de méthodes (noms, paramètres, type de retour), mais pas d'implémentation (pas de code dans le corps des méthodes, en tout cas dans le modèle classique que tu dois retenir pour le moment).

Exemple très simplifié :

```
public interface Vehicule {  
    void demarrer();  
    void arreter();  
}
```

Ensuite une classe "concrète" implémente ce contrat :

```
public class Voiture implements Vehicule {  
    @Override  
    public void demarrer() {  
        // vrai code ici  
    }  
    @Override  
    public void arreter() {  
        // vrai code ici  
    }  
}
```

Idée clé :

L'interface dit : "Tout véhicule doit avoir demarrer() et arreter()".

La classe dit : "Voici comment une voiture fait demarrer() et arreter()".

Dans api/src/main/java/fr/mycellius/repository, crée une interface : WikiRepository.java

```
package fr.mycellius.repository;

import fr.mycellius.domain.WikiPage;
import java.util.List;

public interface WikiRepository {
    WikiPage save(WikiPage page);
    boolean existsById(String id);
    WikiPage getById(String id);
    List<WikiPage> searchByTitleContaining(String fragment);
    List<WikiPage> findAll();
}
```

Idée :

- WikiService ne dépendra plus d'une implémentation concrète (InMemory ou DB), mais de cette interface.

6.2 Faire implémenter l'interface par InMemoryWiki

Dans InMemoryWiki.java, modifie la déclaration de classe :

```
public class InMemoryWiki implements WikiRepository {
```

Les méthodes existent déjà ; il suffit qu'elles correspondent à la signature de l'interface.

Résultat : les tests de séance 3 continuent à utiliser InMemoryWiki, mais via l'interface.

`public class InMemoryWiki implements WikiRepository { ... }` signifie que ta classe "promet" de respecter le contrat défini par l'interface WikiRepository.

Conséquences concrètes :

- Obligation d'implémenter toutes les méthodes de WikiRepository (sinon erreur de compilation,

sauf si la classe est abstract).

- Tu peux utiliser InMemoryWiki partout où un WikiRepository est attendu (polymorphisme), par

exemple en injection Spring :

- > `private final WikiRepository repo;` et Spring peut injecter InMemoryWiki.

- Tu peux remplacer facilement l'implémentation (ex. JpaWikiRepository) sans changer le code qui consomme WikiRepository.

- `public class InMemoryWiki { ... }` signifie que ta classe n'annonce aucun contrat.

Conséquences :

- Aucune obligation d'avoir un ensemble de méthodes précis.

- Elle ne peut pas être utilisée comme un WikiRepository (donc pas injectée dans un champ/constructeur typé WikiRepository).

- Remplacer l'implémentation demandera souvent de modifier le code appelant (car il dépend de la classe concrète).

En une phrase :

avec `implements WikiRepository`, tu programmes "contre une interface" (plus flexible, testable, maintenable) ; sans `implements`, tu programmes "contre une implémentation" (plus rigide).

En gros, Imagine que WikiRepository est une "prise standard" (un format imposé), et InMemoryWiki est un appareil.

Avec implements WikiRepository

InMemoryWiki dit : "Je suis compatible avec la prise WikiRepository."

Donc :

Le compilateur vérifie que InMemoryWiki a bien TOUTES les méthodes exigées par WikiRepository.

Partout où on demande un WikiRepository, on peut brancher InMemoryWiki.

6.3 Créer l'entité JPA pour les pages

Pour éviter de mélanger directement WikiPage (domaine) et la couche de persistance, on crée une entité JPA dédiée.

Dans api/src/main/java/fr/mycellius/persistence, crée WikiPageEntity.java :

```
package fr.mycellius.persistence;

import jakarta.persistence.*;

@Entity
@Table(name = "wiki_page")
public class WikiPageEntity {

    @Id
    private String id;

    @Column(nullable = false, length = 255)
    private String title;

    @Lob
    @Column(nullable = false)
    private String content;

    protected WikiPageEntity() {
        // constructeur sans argument pour JPA
    }

    public WikiPageEntity(String id, String title, String content) {
        this.id = id;
        this.title = title;
        this.content = content;
    }

    // getters / setters classiques
```

```

public String getId() { return id; }
public void setId(String id) { this.id = id; }

public String getTitle() { return title; }
public void setTitle(String title) { this.title = title; }

public String getContent() { return content; }
public void setContent(String content) { this.content = content; }

}

```

Remarque :

- On ne met pas encore les tags dans cette entité. Ils seront traités plus tard (séance 6) via une autre table ou un mapping plus riche.

6.4 Créer le repository Spring Data JPA

Toujours dans fr.mycellius.persistence, crée WikiPageJpaRepository.java :

- Clic droit sur le package fr.mycellius.persistence
- New → Java Class...

Dans la fenêtre :

- Name : WikiPageJpaRepository
- Kind / Type : choisis Interface

IntelliJ va générer quelque chose comme :

```

package fr.mycellius.persistence;

public interface WikiPageJpaRepository {
}

```

Ensuite tu remplaces le contenu par :

```

package fr.mycellius.persistence;

import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface WikiPageJpaRepository extends JpaRepository<WikiPageEntity, String> {

    List<WikiPageEntity> findByTitleContainingIgnoreCase(String fragment);

}

```


=> Spring Data génère automatiquement l'implémentation à partir du nom de la méthode.

6.5 Créer un adapter DatabaseWikiRepository qui implémente WikiRepository

Dans api/src/main/java/fr/mycellius/repository, crée DatabaseWikiRepository.java :

```
package fr.mycellius.repository;

import fr.mycellius.domain.WikiPage;
import fr.mycellius.persistence.WikiPageEntity;
import fr.mycellius.persistence.WikiPageJpaRepository;
import fr.mycellius.domain.exception.PageNotFoundException;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.stream.Collectors;

@Repository
public class DatabaseWikiRepository implements WikiRepository {

    private final WikiPageJpaRepository jpa;

    public DatabaseWikiRepository(WikiPageJpaRepository jpa) {
        this.jpa = jpa;
    }

    @Override
    public WikiPage save(WikiPage page) {
        WikiPageEntity entity = new WikiPageEntity(
            page.getId(),
            page.getTitle(),
            page.getContent()
        );
        WikiPageEntity saved = jpa.save(entity);
        return new WikiPage(
            saved.getId(),
            saved.getTitle(),
            saved.getContent()
        );
    }

    @Override
    public boolean existsById(String id) {
        return jpa.existsById(id);
    }
}
```

```

@Override
public WikiPage getByld(String id) {
    return jpa.findByld(id)
        .map(e -> new WikiPage(e.getId(), e.getTitle(), e.getContent()))
        .orElseThrow(() -> new PageNotFoundException(id));
}

@Override
public List<WikiPage> searchByTitleContaining(String fragment) {
    return jpa.findByTitleContainingIgnoreCase(fragment)
        .stream()
        .map(e -> new WikiPage(e.getId(), e.getTitle(), e.getContent()))
        .collect(Collectors.toList());
}

@Override
public List<WikiPage> findAll() {
    return jpa.findAll()
        .stream()
        .map(e -> new WikiPage(e.getId(), e.getTitle(), e.getContent()))
        .collect(Collectors.toList());
}
}

```

=> Bien sûr, adapte les constructeurs de WikiPage à la signature réelle de l'entreprise.

=> Ce code sert à une chose très précise : brancher ton modèle métier "WikiPage" sur la base de données MySQL, proprement, sans que le reste de ton code connaisse JPA ni la structure technique de la table.

Concrètement, le rôle de DatabaseWikiRepository :

1. C'est une implémentation de l'interface WikiRepository

- Le service métier (WikiService) ne parle qu'en termes de WikiRepository, donc il ne sait pas si derrière on utilise une Map en mémoire (InMemoryWiki) ou une vraie base MySQL (DatabaseWikiRepository).

- Ça permet de changer de stockage sans réécrire toute la logique métier.

2. Il utilise WikiPageJpaRepository (Spring Data JPA) pour parler à la base

- jpa.save(...), jpa.findByld(...), jpa.findAll(), jpa.existsByld(...), jpa.findByTitleContainingIgnoreCase(...) sont les vraies opérations JPA / SQL.

- Ce repository JPA travaille avec des WikiPageEntity qui représentent la table en base.

3. Il fait la traduction entre Entity et Domain

- En entrée : tu reçois une WikiPage (objet métier) → tu construis une WikiPageEntity pour la base.
- En sortie : tu récupères une WikiPageEntity de la base → tu la transformes en WikiPage pour le domaine métier.
- C'est pour ça que tu vois plein de new WikiPage(...) et new WikiPageEntity(...).

4. Il gère l'erreur "page introuvable" côté base

- Dans getByld, si findByld ne trouve rien, on lance une PageNotFoundException(id).
- Donc le contrat métier reste le même que pour InMemoryWiki : "si la page n'existe pas, on lève cette exception".

5. L'annotation @Repository

- Indique à Spring : "cette classe est un composant de type repository, tu peux l'instancier et l'injecter automatiquement".
- Ainsi, Spring peut injecter DatabaseWikiRepository dans WikiService sans que tu fasses new à la main.

Donc, en une phrase :

DatabaseWikiRepository est l'adaptateur entre ton domaine (WikiPage, WikiService, exceptions métier) et la base de données MySQL via Spring Data JPA, en respectant le contrat défini par l'interface WikiRepository.

6.6 Adapter WikiService pour dépendre de WikiRepository

On va faire passer WikiService d'une dépendance "concrète" (InMemoryWiki) à une dépendance "abstraite" (WikiRepository).

Étape 1 — Ouvrir WikiService.java

Fichier à ouvrir dans IntelliJ :

api/src/main/java/fr/mycellius/service/WikiService.java

Étape 2 — Corriger les imports

En haut du fichier, tu as ceci :

```
import fr.mycellius.domain.Tag;
import fr.mycellius.repository.InMemoryWiki;
import fr.mycellius.domain.exception.PageNotFoundException;
import fr.mycellius.domain.WikiPage;
```

On va remplacer l'import de InMemoryWiki par l'import de WikiRepository.

Supprime cette ligne :

```
import fr.mycellius.repository.InMemoryWiki;
```

Ajoute juste en dessous des autres imports :

```
import fr.mycellius.repository.WikiRepository;
```

Résultat attendu dans la zone des imports :

```
import fr.mycellius.domain.Tag;
import fr.mycellius.repository.WikiRepository;
import fr.mycellius.domain.exception.PageNotFoundException;
import fr.mycellius.domain.WikiPage;
```

Étape 3 — Changer le type du champ

Aujourd'hui tu as :

```
private final InMemoryWiki repository;
```

On veut que le service dépende de l'interface WikiRepository, pas de la classe concrète.

Remplace cette ligne par :

```
private final WikiRepository repository;
```

Tu ne touches à rien d'autre pour l'instant.

Étape 4 — Changer le constructeur

Actuellement, tu as :

```
public WikiService(InMemoryWiki repository) {  
    if (repository == null) {  
        throw new IllegalArgumentException("repository obligatoire");  
    }  
    this.repository = repository;  
}
```

On garde la logique, mais on change le type.

Remplace tout le constructeur par :

```
public WikiService(WikiRepository repository) {  
    if (repository == null) {  
        throw new IllegalArgumentException("repository obligatoire");  
    }  
    this.repository = repository;  
}
```

Le reste de la classe (createPage, getPageById, searchByTitle, listAllPages) ne change pas.

Étape 5 — Vérifier que tout compile

Dans IntelliJ, vérifie que tu n'as plus d'erreur rouge dans WikiService.java.

Dans un terminal, à la racine de ton repo mycellius, lance :

```
mvn -f api/pom.xml test
```

Si tout est bien câblé (InMemoryWiki et DatabaseWikiRepository implémentent bien WikiRepository), ça doit passer.

Pourquoi on fait ça ?

Avant : WikiService était “collé” à une implémentation précise (InMemoryWiki = stockage en mémoire).

Après : WikiService parle à une abstraction (WikiRepository), ce qui permet de brancher soit l'implémentation mémoire, soit l'implémentation base de données (DatabaseWikiRepository), sans changer le code métier du service.

Effet :

- Dans les tests unitaires, tu continues à faire :
 - `WikiService service = new WikiService(new InMemoryWiki());`
- Dans l'application Spring Boot, c'est `DatabaseWikiRepository` (annoté `@Repository`) qui sera injecté automatiquement, car c'est l'implémentation de `WikiRepository` gérée par Spring.

Par analogie “magasin / entrepôt”.

Personnages de l'histoire :

- `WikiService` = le vendeur au comptoir du magasin
- `WikiRepository` = le rôle “responsable des stocks” (contrat : ce qu'il doit savoir faire)
- `DatabaseWikiRepository` = la personne réelle qui joue ce rôle de responsable des stocks, mais en travaillant avec un ENTREPÔT réel (la base MySQL)
- `WikiPage` = la fiche produit “propre” utilisée par le vendeur (modèle métier)
- `WikiPageEntity` = le carton dans l'entrepôt, avec l'étiquette au format logistique (modèle base de données)
- `WikiPageJpaRepository` = le système automatique de l'entrepôt (les chariots / tapis roulants gérés par Spring Data JPA)
- MySQL = l'entrepôt physique

Comment ça se passe concrètement :

- Le vendeur (`WikiService`) demande au responsable des stocks (`WikiRepository`) :
 - > “Enregistre cette fiche produit” ou “Donne-moi la fiche du produit PAGE-001”.
- Dans notre cas, le responsable des stocks concret est `DatabaseWikiRepository` :
 - > Il reçoit une `WikiPage` (fiche produit métier côté magasin).
 - > Il la transforme en `WikiPageEntity` (carton au format entrepôt).
 - > Il demande au système automatique (`WikiPageJpaRepository`) de ranger ou de chercher ce carton dans l'entrepôt (MySQL).
- Quand l'entrepôt répond avec un carton (`WikiPageEntity`), `DatabaseWikiRepository` le re-transforme en fiche produit (`WikiPage`) pour le vendeur.
- Si l'entrepôt ne trouve pas le carton demandé, `DatabaseWikiRepository` dit : “Produit introuvable” → `PageNotFoundException`.

Donc, l'idée clé :

- Le vendeur (WikiService) ne voit jamais l'entrepôt ni les cartons.
- Il parle toujours en WikiPage.
- DatabaseWikiRepository est le traducteur entre le monde "magasin métier" (WikiPage, exceptions métier) et le monde "entrepôt technique" (MySQL, JPA, WikiPageEntity).

6.7 Vérifier que les endpoints C1/C2 utilisent bien la DB

Redémarre l'appli :

```
mvn -f api/pom.xml spring-boot:run
```

Dans Postman (ou avec curl) :

- Créer une page :

```
curl -X POST http://localhost:8080/api/v1/pages
```

```
-H "Content-Type: application/json"
```

```
-d '{"id":"PAGE-DB-001","title":"Page persistée","content":"Contenu DB"}'
```

- Lire la page :

```
curl http://localhost:8080/api/v1/pages/PAGE-DB-001
```

- Arrête l'appli (Ctrl+C), redémarre-la, puis refais le GET :

```
curl http://localhost:8080/api/v1/pages/PAGE-DB-001
```

=> Si tout est correct, la page est toujours là : la persistance fonctionne.

7. Sauvegarde / restauration de la DB (dev)

Objectif

Montrer qu'on peut sauvegarder l'état de la DB puis le restaurer, ce qui sera crucial plus tard pour les séances "prod" et la cybersécurité.

7.1 Préparer un dossier de backups

Dans ton dépôt, crée par exemple **infra/db/backups**.

Précondition de test:

Objectif :

Pour disposer d'au moins une donnée "repère" (une page) dont l'existence est vérifiée avant la backup, afin de pouvoir prouver ensuite que la restauration a bien fonctionné.

Pré-requis techniques (à vérifier maintenant, sinon le test n'a pas de sens)

A. Le conteneur MySQL "mycellius-mysql" doit tourner.

B. L'API Spring Boot doit tourner sur <http://localhost:8080>

C. Postman est ouvert (collection "Mycellius – Local API" de la séance 4 si tu l'as).

Étape 1 — Vérifier que MySQL (Docker) est bien démarré

- Ouvre un terminal (PowerShell ou terminal IntelliJ).
- Place-toi à la racine du dépôt (chez toi : C:\Users\clakg\mycellius) puis démarre MySQL :

```
cd C:\Users\clakg\mycellius
cd infra/db
docker compose up -d
docker ps
```

Résultat attendu : tu vois un conteneur mycellius-mysql avec le port 3306 exposé.

Étape 2 — Démarrer l'API Spring Boot (dans un 2e terminal, séparé du terminal Docker)

- Ouvre un DEUXIÈME terminal (ne coupe pas le premier).
- Place-toi à la racine du dépôt, puis lance l'API :

```
cd C:\Users\clakg\mycellius
mvn -f api/pom.xml spring-boot:run
```

- Résultat attendu : dans les logs, tu dois voir une ligne de type "Started ..." et "Tomcat started on port 8080". L'API doit rester "en cours d'exécution" (ne ferme pas ce terminal).

(Option de vérification rapide, si tu as l'endpoint de santé)

Dans un navigateur :

<http://localhost:8080/api/health>

Résultat attendu : statut UP.

Étape 3 — Créer une page via Postman (comme en séance 4, mais en /api/v1/pages)

- Dans Postman :

(Recommandé) Vérifie que ta variable d'environnement existe :

baseUrl = <http://localhost:8080>

- Crée une requête Postman :

- > Nom : PRECOND – Create page (DB)

- > Méthode : POST

- > URL : {{baseUrl}}/api/v1/pages

- > Onglet Body → raw → JSON

- > Colle ce JSON :

```
{
  "id": "PAGE-DB-001",
  "title": "Page persistée (précondition backup)",
  "content": "Cette page doit exister AVANT le mysqldump, pour être relue APRÈS la restauration.",
  "tags": []
}
```

- Clique sur Send.

- Résultat attendu : succès (souvent 201) + retour JSON de la page créée.

Étape 4 — Noter l'id (obligatoire)

Note dans ton compte-rendu l'id exact utilisé :

> PAGE-DB-001

Étape 5 — Faire un GET immédiat sur le même id pour confirmer qu'elle existe

Dans Postman, crée une deuxième requête :

- > Nom : PRECOND – Get page (DB)
- > Méthode : GET
- > URL : {{baseUrl}}/api/v1/pages/PAGE-DB-001
- > Clique sur Send.

Résultat attendu : 200 OK + JSON de la page PAGE-DB-001.

Conclusion (très important)

Quand tu as :

- un POST réussi sur /api/v1/pages
- puis un GET réussi sur /api/v1/pages/PAGE-DB-001

=> alors seulement après tu peux passer à 7.2 "Faire un backup avec mysqldump". Sinon, tu risques de dumper une base vide (ou pas la bonne base) et tu ne pourras pas valider la restauration.

7.2 Faire un backup avec mysqldump

Assure-toi que le conteneur tourne et que tu as quelques pages créées.

Depuis la racine du dépôt :

```
cd infra/db
```

```
docker exec -i mycellius-mysql mysqldump -u root -p rootpassword mycellius > backups/  
mycellius_backup.sql
```

Commentaires :

- **-u root -prootpassword** : utilise le compte root défini dans docker-compose.
- le fichier **backups/mycellius_backup.sql** contient tout le schéma + les données de la DB mycellius.

7.3 Tester une restauration simple

Attention : cette partie est à faire sur la DB de développement uniquement.

Toujours dans infra/db :

```
docker exec -i mycellius-mysql mysql -u root -prootpassword -e "DROP DATABASE mycellius;  
CREATE DATABASE mycellius;"
```

```
docker exec -i mycellius-mysql mysql -u root -prootpassword mycellius < backups/  
mycellius_backup.sql
```

Vous allez surement avoir un bug... Indices : configuration de terminal IntelliJ... Faites moi signe quand vous êtes arrivés là

Puis :

- redémarre l'appli Spring Boot si nécessaire
- refais un GET sur une page que tu avais créée avant le backup comme /api/health :
 - si elle est encore là, la restauration est validée.

Idée :

- Tu relis le concept de disponibilité / intégrité (CIA) de la séance 1 avec une vraie manip technique.
- tu vois que le backup/restore n'est pas juste de la théorie.

8. CI + workflow Git (dev → test) avec persistance DB

Objectif

Garder le rituel propre : le code peut devenir plus complexe (DB, JPA), mais le workflow Git + CI reste le même.

8.1 Sur ton poste (ligne de commande, branche dev)

```
git checkout dev
```

```
git pull origin dev
```

Coder dans api/ (avec IntelliJ) :

- modifications dans pom.xml (dépendances JPA, Flyway, MySQL)
- application.properties (config DB + Flyway)
- nouvelles classes :
 - WikiPageEntity,
 - WikiPageJpaRepository,
 - WikiRepository,
 - DatabaseWikiRepository
- adaptation d'InMemoryWiki et de WikiService

Vérifier localement :

```
mvn -f api/pom.xml test
```

Important :

- Les tests unitaires doivent continuer à passer sans nécessiter une DB (ils utilisent InMemoryWiki).
- On n'ajoute pas encore de tests d'intégration Spring qui touchent la vraie DB dans la CI (ce sera pour plus tard).

Avant d'aller plus loin...

les modifications à la séance 5 ne te permettent pas de faire la procédure habituelle pour merge dev sur test sur ton terminal et ton git add plante... oui je sais c'est relou...

Pourquoi ça t'arrive dans la séance 5 ?

Dans ton docker-compose.yml, tu as (comme proposé dans le TP) un bind-mount ./data:/var/lib/mysql. Donc MySQL écrit ses fichiers internes directement dans infra/db/data/... et Git essaie de tout ajouter dès que tu fais git add infra/db/.

Même problème côté api/target/ : ce sont des artefacts Maven (compilation + rapports de tests) qui ne doivent jamais partir dans Git.

Ce qu'il faut faire

1. Ajouter dans le .gitignore mycellius/.idea/.gitignore

À la racine du repo, dans .gitignore, ajoute au minimum :

```
# IntelliJ / IDE
.idea/
*.iml
```

```
# Maven / Java build
**/target/
```

```
# MySQL (Docker) - données locales (NE PAS VERSIONNER)
infra/db/data/
infra/db/backups/
```

Note : j'ignore aussi infra/db/backups/ car tes dumps sont des données locales de dev (et potentiellement sensibles). La séance 5 demande de tester backup/restore, pas de versionner la donnée.

2. Si tu avais déjà "tracké" ces dossiers (cas possible), les retirer de l'index

Exécute :

```
git rm -r --cached api/target infra/db/data .idea 2> /dev/null || true
```

Préparer le commit :

```
git status
```

```
git add api/ infra/db/
```

```
git commit -m "feat(db): persist WikiPage with MySQL + JPA + Flyway"
```

Pousser sur dev :

```
git push origin dev
```

8.2 Sur GitHub

- onglet Actions : vérifier que la CI "CI" est verte sur la branche dev
- créer une Pull Request :
 - base : test
 - compare : dev
- vérifier que les checks CI sont verts

- merger la PR vers test si tout est OK

Optionnel :

- créer un tag v0.2.0 après merge sur test pour marquer “Mycellius après séance 5 (API Spring Boot branchée sur MySQL avec migrations)”.

Conclusion de la séance 5

À ce stade, tu as :

- une API REST Spring Boot fonctionnelle, déjà construite en séance 4
- un schéma MySQL minimal pour les pages, piloté par des migrations Flyway versionnées
- un repository basé sur JPA/Hibernate qui implémente le même contrat que le dépôt en mémoire
- des endpoints C1/C2 qui lisent et écrivent réellement dans une base de données persistante
- une mini procédure de backup/restore sur l’environnement de développement
- une CI qui reste simple (tests unitaires en mémoire) mais qui sécurise toute la couche métier

La séance 6 pourra s’appuyer directement sur cette base pour :

- introduire des DTO “propres” pour l’API (entrée/sortie)
- ajouter de la validation (contraintes sur les DTO)
- améliorer la gestion standardisée des erreurs
- enrichir l’API côté lecture (pagination, recherche) et côté tags en s’appuyant sur la DB déjà en place.